SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: **Theory of Computation** **COURSE CODE:** **CSA13**

COURSE OUTCOME COVERED

CO2: De**ign Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Duration : 1 Hour

Design Task

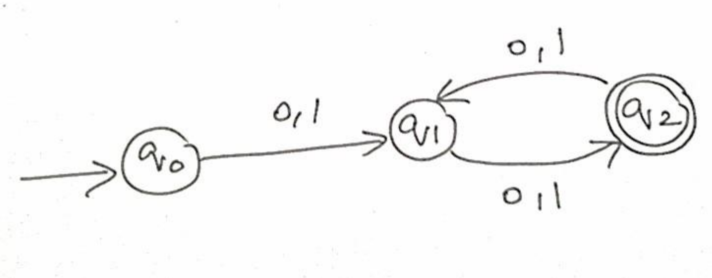
Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Design the regular expression for the language represented by the given automata



2. Consider the following grammar

S->aSc|T|U|e

T-> bTc|e

U->aUb|e

A} What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

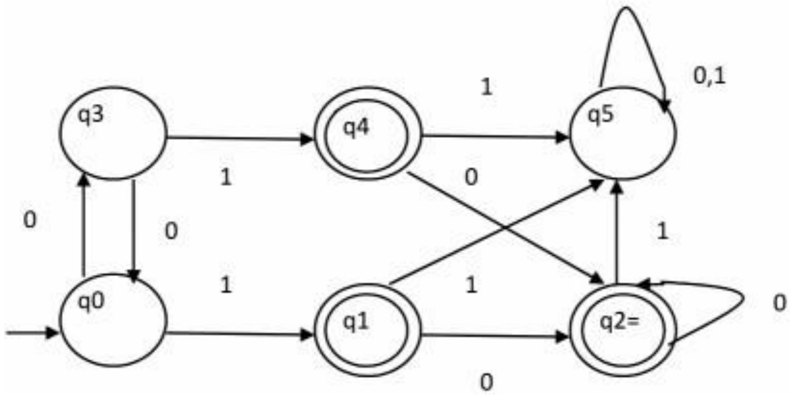
C) A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

3. Design a non deterministic finite automata with ε-transitions to represent the language denoted by the regular expression
00*1+11*0

4. **I)**Show that if L1 and L2 are regular then (L1+l2) also regular

Ii)Is the language { a^p| p is prime} regular? Justify

5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states



RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	**10 Marks**	/10	/10	/10	/10	/10	**/ 50**

100

1. Smart Security System for Research Laboratory

- a → valid ID Scan
- c → Door access confirmation
- b → biometric verification

Security rules states that start with a, end with c

Examples of valid sequences : ac, abc, abbc, abbbc.

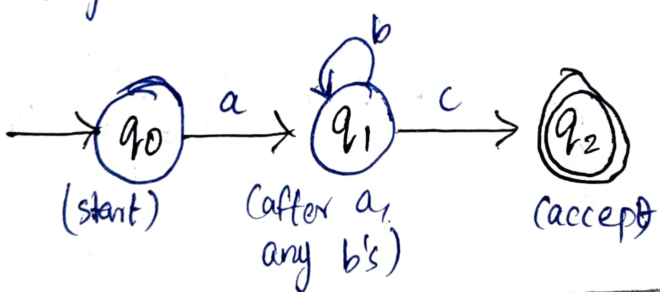
RE : ab^*c

Transition Table

State	a	b	c
q_0	q_1	–	–
q_1	–	q_1	q_2
q_2	–	–	–

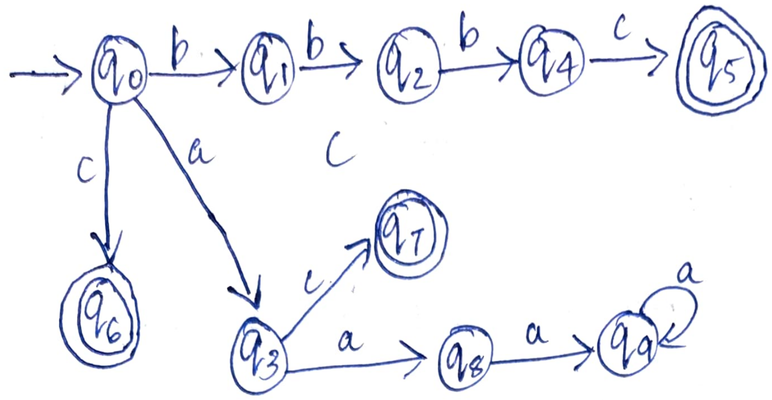
q_0 : start state
q_2 : final state

DFA diagram for ab^*c



2. Given a DFA used for validating a specific PIN pattern bbbc, bca, c, ca, caaaaaa
($\Sigma = \{a, b, c\}$)

DFA for the pattern set.



Final states : q_5 (bbbc), q_7 (bca), q_6 (c), q_3 (ca), q_9 (caaaaaa)

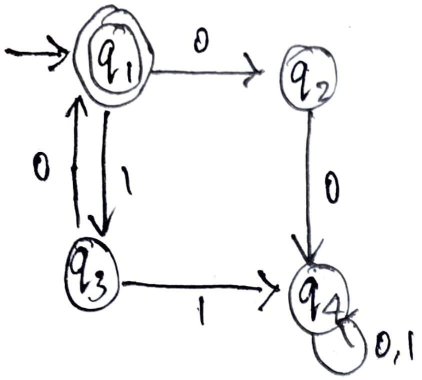
Minimization of DFA

Step	Partitions
0 (by final / non-final)	$\{q_5, q_6, q_7, q_3, q_9\}$, $\{q_0, q_1, q_2, q_4, q_8\}$
1	$\{q_5, q_7, q_6, q_3, q_9\}$, $\{q_0\}$, $\{q_1, q_8\}$, $\{q_2\}$, $\{q_4\}$
2 (refine $\{q_1, q_8\}$)	$\{q_5, q_7, q_6, q_3, q_9\}$, $\{q_0\}$, $\{q_1\}$, $\{q_8\}$, $\{q_2\}$, $\{q_4\}$
3 (all distinct)	All states are pairwise distinguishable

How state reduction improves performance in embedded systems.

- Fewer states → less memory required to store state table
- Fewer transitions → faster decision making and less computation
- Lower power consumed → important for battery-operated devices.
- Smaller hardware → Reduces chip area and cost
- Easier verification and maintenance of the system.

3. Given finite automation into regular expression (State Elimination Method)



Eliminate q_2

$q_1 \rightarrow q_4$: 0 0 = 00

(also $q_3 \rightarrow q_4$ remains 1)

Other transitions changed.

Eliminate q_3

$q_1 \rightarrow q_4$ gets new path ~~graph~~

$q_1 \rightarrow q_3 \rightarrow q_4 = 1\ 1 = 11$

Existing $q_1 \rightarrow q_4 = 00$

So $q_1 \rightarrow q_4 = 00 + 11$

q_4 loop $= 0+1$

Eliminate q_4

Start q_1, Final q_1

$R = (00+11)(0+1)^*$

Regular Expression: $(00+11)(0+1)^*$

4. Consider following grammar

$S \rightarrow aSc \mid T \mid U \mid \varepsilon$

$T \rightarrow bTc \mid \varepsilon$

$U \rightarrow aUb \mid \varepsilon$

A) Language defined by the grammar

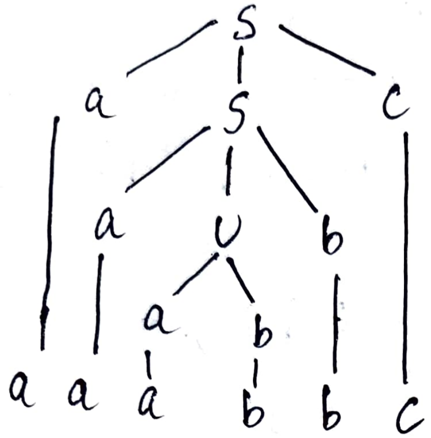
From $T \rightarrow bTc \mid \varepsilon \Rightarrow L(T) = \{b^n c^n \mid n \geq 0\}$

From $U \rightarrow aUb \mid \varepsilon \Rightarrow L(U) = \{a^n b^n \mid n \geq 0\}$

From $S \rightarrow aSc \mid T \mid U \mid \varepsilon$

$\Rightarrow L(S) = \{a^i b^j c^{i+j} \mid i,j \geq 0\} \cup \{a^{i+j} b^j c^i \mid i,j \geq 0\}$

B) Derivation Tree



c) Yes the grammar is ambiguous.

Consider the string ac

Derivation 1:

$S \Rightarrow aSc \Rightarrow ac$ (using $S \Rightarrow \varepsilon$)

Derivation 2:

$S \Rightarrow aSc \Rightarrow aTc \Rightarrow ac$

(using $S \Rightarrow T$ and $T \Rightarrow \varepsilon$)

Since ac has two different leftmost derivations, the grammar is ambiguous.

5. If L1 and L2 are regular then (L1+L2) also regular
[Closure under Union)

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ accepts L1

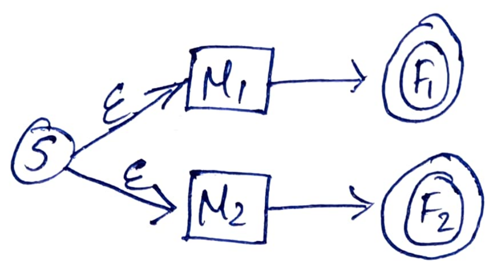
and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ accepts L2.

Construct NFA M:

- New start state: s
- ε-transition from s to q_1 & q_2
- Final states: $F = F_1 \cup F_2$

Then $L(M) = L_1 \cup L_2$, which is regular.

NFA for $L_1 \cup L_2$



ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular?

No, it is not regular.

(Using Pumping Lemma)

Assume L is regular with pumping length p.

Chose a prime $q > p$ and let $w = a^q \in L$.

For any decomposition $w = xyz$ with $|xy| \leq p$ & $|y| > 0$,

$y = a^k$ for some $k \geq 1$

Pump y up ($i = q+1$): $xy^{q+1}z = a^{q+kq} = a^{q(1+k)}$

Length $= q(1+k)$, which is composite (since q and $1+k > 1$).

Thus $xy^{q+1}z \notin L$, contradiction.

$\therefore$, $\{a^p \mid p \text{ is prime}\}$ is not regular.